

Object Detection in PyTorch

Objectives:

- Understanding different algorithms for object detection
- Implementing RCNN and FasterRCNN for object detection
- Testing on test data.

What is Object Detection?

Object detection is a [computer vision technique](#) in which a software system can detect, locate, and trace the object from a given image or video. The special attribute about object detection is that it identifies the class of object (person, table, chair, etc.) and their location-specific coordinates in the given image. The location is pointed out by drawing a bounding box around the object. The bounding box may or may not accurately locate the position of the object. The ability to locate the object inside an image defines the performance of the algorithm used for detection.

How is object detection different from object classification?

Object classification is a traditional computer vision task that is effectively determining the class of the object in an image. Object classification finds out what the object in a given picture or video is. There is a probability score associated with the results so that we can get the confidence scores of the results.

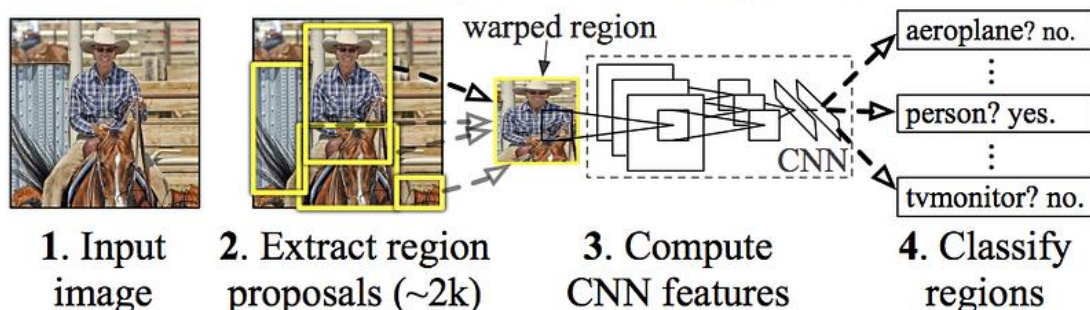
1. Region-based Convolutional Neural Networks(R-CNN):

Since we know that object detection is a classification problem, the success of the model depends on the accuracy of the classification of all objects. The general idea is to use CNNs. But a problem with CNN's is that they are too slow and computationally expensive. Hence it's not feasible to run CNNs on so many patches generated by sliding window detectors.

Hence, R-CNN we introduced. R-CNN networks solve this problem by using an object proposal algorithm termed Selective Search which is used to reduce the number of bounding boxes that are being fed to the classifier to a maximum of 2000 region proposals. Selective search uses features like texture, pixel intensity, color, etc to generate all possible locations of objects in an image. Now, these boxes can be fed to our CNN based classifier. So we run Selective Search to generate probable objects.

- These patches are then fed to CNN, followed by an SVM classifier to predict the class of objects in each patch.
- We then optimize all patches by training a model for bounding box regression exclusively.

R-CNN: Regions with CNN features

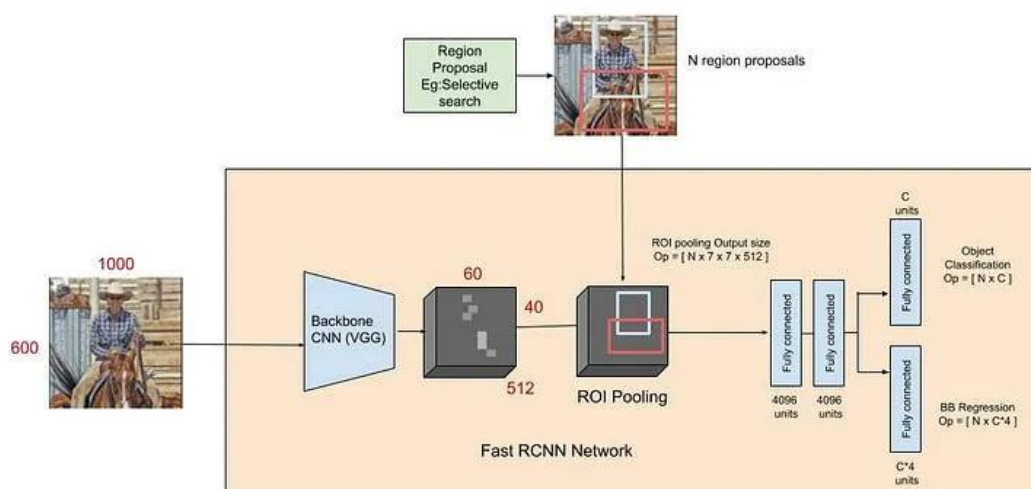


2. Fast R-CNN:

Fast R-CNN was introduced because R-CNN architectures were very slow. Fast RCNN uses the concepts of RCNN. But it has a few architectural changes as compared to R-CNN architectures. Instead of processing 2,000 separate regions, it passes the *entire* image through the CNN once to create a feature map, then crops regions from that map. This significantly speeded up detection.

In Fast R-CNN architectures the bounding box regression was added to the neural network training instead of doing it separately. It enabled the network to have two heads, classification head, and bounding box regression head.

These two changes reduced the overall training time and increased the accuracy.



3. Faster R-CNN:

An improvement over Fast R-CNN was faster R-CNN. Replaced the slow Selective Search with a **Region Proposal Network (RPN)** integrated directly into the model. This allowed the network to propose its own regions, enabling near real-time performance.

Apart from that, we have some more networks which are very popular.

1. Yolo
2. SSD

Setup Environment:

First, install the required Python libraries. Open your terminal or command prompt and run:

```
pip install torch torchvision Pillow matplotlib
```

Verify Installation:

Open a Python shell and confirm everything is installed:

```
import torch
import torchvision
from PIL import Image
import matplotlib.pyplot as plt

print(f"PyTorch version: {torch.__version__}")
print(f"Torchvision version: {torchvision.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

Understand the COCO class labels:

The pre-trained model recognizes 80 object categories from the COCO dataset. Create a list of these labels in your script. The model outputs a number (1–90), and we map it to a human-readable name.

```
COCO_LABELS = [
    '__background__', 'person', 'bicycle', 'car',
    'motorcycle', 'airplane', 'bus', 'train', 'truck',
    'boat', 'traffic light', 'fire hydrant', 'N/A',
    'stop sign', 'parking meter', 'bench', 'bird',
    'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant',
    'bear', 'zebra', 'giraffe', 'N/A', 'backpack',
    'umbrella', 'N/A', 'N/A', 'handbag', 'tie',
    'suitcase', 'frisbee', 'skis', 'snowboard',
    'sports ball', 'kite', 'baseball bat',
```

```
'baseball glove', 'skateboard', 'surfboard',  
'tennis racket', 'bottle', 'N/A', 'wine glass',  
'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana',  
'apple', 'sandwich', 'orange', 'broccoli', 'carrot',  
'hot dog', 'pizza', 'donut', 'cake', 'chair',  
'couch', 'potted plant', 'bed', 'N/A',  
'dining table', 'N/A', 'N/A', 'toilet', 'N/A',  
'tv', 'laptop', 'mouse', 'remote', 'keyboard',  
'cell phone', 'microwave', 'oven', 'toaster',  
'sink', 'refrigerator', 'N/A', 'book', 'clock',  
'vase', 'scissors', 'teddy bear', 'hair drier',  
'toothbrush'  
]
```

Load the pre-trained model:

Load the Faster R-CNN model with pre-trained weights. The model will automatically download the weights the first time you run this (about 160 MB).

```
from torchvision.models.detection import \  
    fasterrcnn_resnet50_fpn_v2, \  
    FasterRCNN_ResNet50_FPN_V2_Weights  
  
# Load model with the best available weights  
weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT  
model = fasterrcnn_resnet50_fpn_v2(weights=weights)  
  
# Set to evaluation mode (important!)  
model.eval()  
  
print("Model loaded successfully!")
```

Load and Preprocess an Image:

Load any image from your computer. The model expects a tensor with pixel values scaled between 0 and 1.

```
from torchvision import transforms  
  
# Load your image (change the path to your image)  
image_path = "test_image.jpg"  
image = Image.open(image_path).convert("RGB")  
  
# Convert image to tensor (scales pixels to 0-1)
```

```
transform = transforms.ToTensor()
image_tensor = transform(image)

print(f"Image size: {image.size}")
print(f"Tensor shape: {image_tensor.shape}")
```

Run Object Detection:

Pass the image tensor through the model. The model returns predictions including bounding boxes, labels, and confidence scores.

```
# Run detection (no gradient needed for inference)
with torch.no_grad():
    predictions = model([image_tensor])

# The model returns a list; get first image's results
pred = predictions[0]

print(f"Detected {len(pred['boxes'])} objects")
print(f"Keys: {pred.keys()}")
```

Understanding the Output:

The model returns a dictionary with three keys:

Key	What It Contains
boxes	Coordinates [x1, y1, x2, y2] for each detection
labels	Integer class ID for each detection
scores	Confidence score (0 to 1) for each detection

Filter Results By Confidence:

The model often returns many low-confidence detections. Filter them using a threshold (0.5 is a good starting point):

```
# Set confidence threshold
THRESHOLD = 0.5

# Filter predictions
boxes = pred['boxes'][pred['scores'] > THRESHOLD]
labels = pred['labels'][pred['scores'] > THRESHOLD]
scores = pred['scores'][pred['scores'] > THRESHOLD]

# Print results
print(f"\nObjects detected (confidence > {THRESHOLD}):\n")
for i in range(len(boxes)):
```

```
label_name = COCO_LABELS[labels[i]]
confidence = scores[i].item() * 100
box = boxes[i].tolist()
print(f"  {label_name}: {confidence:.1f}%")
print(f"    Location: ({box[0]:.0f}, {box[1]:.0f})"
      f" to ({box[2]:.0f}, {box[3]:.0f})")
```

Visualize the Detections:

Now let's draw the bounding boxes and labels on the image using matplotlib:

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import random

def visualize_detections(image, boxes, labels,
                        scores, threshold=0.5):
    fig, ax = plt.subplots(1, figsize=(12, 8))
    ax.imshow(image)

    for i in range(len(boxes)):
        box = boxes[i].tolist()
        label = COCO_LABELS[labels[i]]
        score = scores[i].item()

        # Random color for each box
        color = [random.random() for _ in range(3)]

        # Draw bounding box
        x1, y1, x2, y2 = box
        width = x2 - x1
        height = y2 - y1
        rect = patches.Rectangle(
            (x1, y1), width, height,
            linewidth=2, edgecolor=color,
            facecolor='none'
        )
        ax.add_patch(rect)

        # Add label text
        ax.text(
            x1, y1 - 5,
            f"{label}: {score:.1%}",
            color='white', fontsize=10,
            fontweight='bold',
            bbox=dict(facecolor=color, alpha=0.7)
```

```
)  
  
ax.axis('off')  
plt.title('Object Detection Results')  
plt.tight_layout()  
plt.savefig('detection_result.jpg', dpi=150)  
plt.show()  
print("Result saved as detection_result.jpg")  
  
# Call the function  
visualize_detections(image, boxes, labels, scores)
```

Lab Task:

1. **Change the threshold:** Try $\text{THRESHOLD} = 0.3$ and $\text{THRESHOLD} = 0.8$. How does it affect the number and accuracy of detections?
2. **Try different images:** Test with a selfie, a street photo, a photo of your desk, and an outdoor scene. Which objects does the model detect best?
3. **Count specific objects:** Write code to count how many "person" or "car" objects appear in an image.
4. **Process multiple images:** Modify the script to loop through all images in a folder and save detection results for each one.
5. **Try another model:** Replace Faster R-CNN with SSD or RetinaNet from torchvision and compare speed vs. accuracy.